JOBSHEET 7
LOOPS 1

1. Objectives

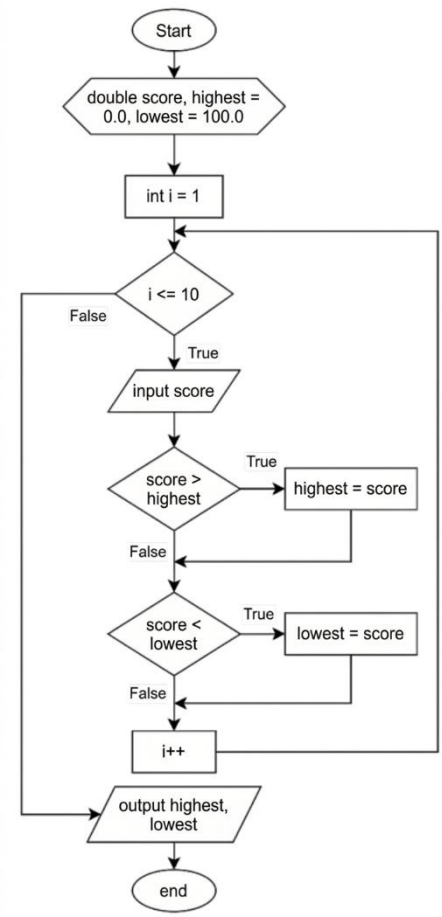
- Students are able to explain the syntax for writing loop programs (for, while, and do-while).
- Students are able to implement loop flowcharts using the Java programming language.

2. Practical Exercises

2.1 Experiment 1: Student Grades Case Study in SIAKAD – FOR Loop

Experiment Duration: 90 minutes

In the Academic Information System (SIAKAD), a lecturer enters the grades of the Programming Fundamentals Practicum course taken by students. The lecturer wants to find the highest and lowest quiz scores among 10 students in one class. The lecturer must enter the score of each student, and then determine and display the highest and lowest scores. Refer to the following flowchart:



Based on the flowchart, write a program using the Java programming language.

2.1.1 Experimental Steps

1. Create a new repository in your GitHub account and name it **programming-fundamentals-jobsheet7**
2. Clone the repository using the **git clone** command from the terminal
3. Open the repository folder using Visual Studio Code
4. Create a new file and name it **SiakadForAttendanceNo.java**
5. Create the basic structure of a Java program consisting of the main() method.
6. Import the Scanner library at the top of the file (outside the class)
7. Declare a Scanner object named **sc** inside the main() method
8. Declare the variables **score**, **highest**, and **lowest** of type double. Initialize **highest** to 0 and **lowest** to 100

```
double score, highest = 0, lowest = 100;
```

9. Create a FOR loop with a condition limit that matches the number of students, which is 10

```
for (int i = 1; i <= 10; i++) {

}
```

10. Inside the FOR loop, add a statement to input the student's **score**. Then, create two separate selection statements to check the highest and lowest scores by comparing the input **score** with the variable **highest** and the variable **lowest**

```
for (int i = 1; i <= 10; i++) {
    System.out.print("Enter score for student " + i + ": ");
    score = sc.nextDouble();
    if (score > highest) {
        highest = score;
    }
    if (score < lowest) {
        lowest = score;
    }
}
```

11. Outside the FOR loop, display the highest and lowest scores

```
System.out.println("Highest score: " + highest);
System.out.println("Lowest score: " + lowest);
```

12. Compile and run the program
13. **Commit and push the program code to GitHub**

2.1.2 Verification of Experimental Results

Compare the output of your program with the following figure.

```
Enter score for student 1: 76.5
Enter score for student 2: 82.3
Enter score for student 3: 62.1
Enter score for student 4: 88.4
Enter score for student 5: 65.9
Enter score for student 6: 67.9
Enter score for student 7: 90.1
```

```
Enter score for student 8: 55.3
Enter score for student 9: 73.7
Enter score for student 10: 78.6
Highest score: 90.1
Lowest score: 55.3
```

2.1.3 Questions

1. Identify and indicate each component of the FOR loop in the program code of Experiment 1!
2. Why is the variable **highest** initialized to 0 and the variable **lowest** initialized to 100? What happens if **highest** is initialized to 100 and **lowest** is initialized to 0?
3. Explain the function and workflow of the following code snippet!

```
if (score > highest) {
    highest = score;
}
if (score < lowest) {
    lowest = score;
}
```

4. Modify the program to count how many students pass and how many students fail, based on the passing threshold (minimum score of 60). Display the number of students who pass and fail after displaying the highest and lowest scores!
5. **Commit and push the program code to GitHub**

2.2 Experiment 2: Student Grades Case Study in SIAKAD – WHILE Loop

Experiment Duration: 90 minutes

A lecturer wants to enter the scores of several students into SIAKAD in order to determine their letter grades. The program must prompt the lecturer to enter the score of each student. If the lecturer enters an invalid score (negative or greater than 100), the program must ignore the input and ask the lecturer to re-enter it. Valid scores are then classified into letter grades: A (80 < score ≤ 100), B+ (73 < score ≤ 80), B (65 < score ≤ 73), C+ (60 < score ≤ 65), C (50 < score ≤ 60), D (39 < score ≤ 50), and E (score ≤ 39).

Based on this case study, write a program using the Java programming language.

2.2.1 Experimental Steps

1. Create a new file and name it **SiakadWhileAttendanceNo.java**
2. Create the basic structure of a Java program consisting of the main() method.
3. Import the Scanner library at the top of the file (outside the class)
4. Declare a Scanner object named **sc** inside the main() method
5. Declare the variables **score**, **studentCount**, and **i** (for the loop) of type integer. Initialize **i** to 0 as the initial value of the loop

6. Write the code to read the number of students and store it in the variable **studentCount**. As a result, the loop limit is dynamic and depends on the user's keyboard input.

```
System.out.print("Enter the number of students: ");
studentCount = sc.nextInt();
```

7. Create a WHILE loop with a condition limit that matches the number of students (for example, 5). Note that the operator used is < because the variable **i** starts at 0, not 1

```
while (i < studentCount) {

    i++;
}
```

8. Inside the WHILE loop, add a statement to input the student's **score**. Then, create an IF selection statement to check whether the entered score is valid, with the requirement that the score must be in the range of 0 to 100. Next, add an IF-ELSE IF-ELSE selection structure to display the letter grade according to the given criteria.

```
while (i < studentCount) {
    System.out.print("Enter score for student " + (i + 1) + ": ");
    score = sc.nextInt();

    if (score < 0 || score > 100) {
        System.out.println("Invalid score. Please enter a valid score!");
        continue;
    }

    if (score > 80 && score <= 100) {
        System.out.println("The grade of student " + (i + 1) + " is A");
    } else if (score > 73 && score <= 80) {
        System.out.println("The grade of student " + (i + 1) + " is B+");
    } else if (score > 65 && score <= 73) {
        System.out.println("The grade of student " + (i + 1) + " is B");
    } else if (score > 60 && score <= 65) {
        System.out.println("The grade of student " + (i + 1) + " is C+");
    } else if (score > 50 && score <= 60) {
        System.out.println("The grade of student " + (i + 1) + " is C");
    } else if (score > 39 && score <= 50) {
        System.out.println("The grade of student " + (i + 1) + " is D");
    } else {
        System.out.println("The grade of student " + (i + 1) + " is E");
    }
    i++;
}
```

9. Compile and run the program
10. **Commit and push the program code to GitHub**

2.2.2 Verification of Experimental Results

Compare the output of your program with the following figure.

```
Enter the number of students: 5
Enter score for student 1: 85
The grade of student 1 is A
Enter score for student 2: 63
```

```
The grade of student 2 is C+
Enter score for student 3: 101
Invalid score. Please enter a valid score!
Enter score for student 3: 23
The grade of student 3 is E
Enter score for student 4: -15
Invalid score. Please enter a valid score!
Enter score for student 4: 70
The grade of student 4 is B
Enter score for student 5: 55
The grade of student 5 is C
```

2.2.3 Questions

1. In the following code snippet, determine the meaning and purpose of the following syntax:

```
if (score < 0 || score > 100) {
    System.out.println("Invalid score. Please enter a valid score!");
    continue;
}
```

   a. **score < 0 || score > 100**
   b. **continue**

2. Why is the **i++** statement written at the end of the WHILE loop? What happens if it is written at the beginning of the WHILE loop?
3. If the number of students entered is 19, how many times will the WHILE loop run?
4. Modify the program so that, if a student receives grade A, the program displays an additional message “Well done, keep up the good work!”
5. **Commit and push the program code to GitHub**

2.3 Experiment 3: Café Transaction Case Study – DO-WHILE Loop

Experiment Duration: 60 minutes

At a café, a cashier wants to process the transactions of several customers. A customer may buy more than one item (coffee at Rp 12,000, tea at Rp 7,000, and bread at Rp 20,000), and the cashier will continue to enter the purchase quantities for each customer. If a customer decides to cancel the transaction (by entering “cancel”), the cashier stops entering transactions and the program terminates.

Based on this case study, write a program using the Java programming language.

2.3.1 Experimental Steps

1. Create a new file and name it **CafeDoWhileAttendanceNo.java**
2. Create the basic structure of a Java program consisting of the main() method.
3. Import the Scanner library at the top of the file (outside the class)
4. Declare a Scanner object named **sc** inside the main() method
5. Declare the variables coffee, tea, and **bread** of type integer to store the quantity of each item purchased by the customer, **customerName** of type String, and **totalPrice**

of type integer to store the total payment. Declare and initialize **coffeePrice** to 12000, **teaPrice** to 7000, and **breadPrice** to 20000.

6. Create a DO-WHILE loop with the condition **true**

```
do {

} while (true);
```

7. Inside the DO-WHILE loop, add a statement to input **customerName**. Then add an IF statement to check the contents of the variable **customerName**. Next, add statements to input the quantity of each menu item purchased by the customer, provided that the customer name entered is not "cancel". Calculate the total purchase price and display the result.

```
do {
    System.out.print("Enter customer name (type 'cancel' to exit): ");
    customerName = sc.nextLine();
    if (customerName.equalsIgnoreCase("cancel")) {
        System.out.println("Transaction canceled.");
        break;
    }
    System.out.print("Coffee quantity: ");
    coffee = sc.nextInt();
    System.out.print("Tea quantity: ");
    tea = sc.nextInt();
    System.out.print("Bread quantity: ");
    bread = sc.nextInt();
    totalPrice = (coffee * coffeePrice) + (tea * teaPrice) + (bread * breadPrice);
    System.out.println("Total amount to pay: Rp " + totalPrice);
    sc.nextLine();
} while (true);

System.out.println("All transactions completed.");
```

Note: sc.nextLine(); after printing totalPrice is a statement used to clear the newline character from the input buffer

8. Compile and run the program
9. **Commit and push the program code to GitHub**

2.3.2 Verification of Experimental Results

Compare the output of your program with the following figure.

```
Enter customer name (type 'cancel' to exit): Rena
Coffee quantity: 3
Tea quantity: 0
Bread quantity: 1
Total amount to pay: Rp 56000
Enter customer name (type 'cancel' to exit): Yuni
Coffee quantity: 1
Tea quantity: 4
Bread quantity: 2
Total amount to pay: Rp 80000
Enter customer name (type 'cancel' to exit): CANCEL
Transaction canceled.
All transactions completed.
```

2.3.3 Questions

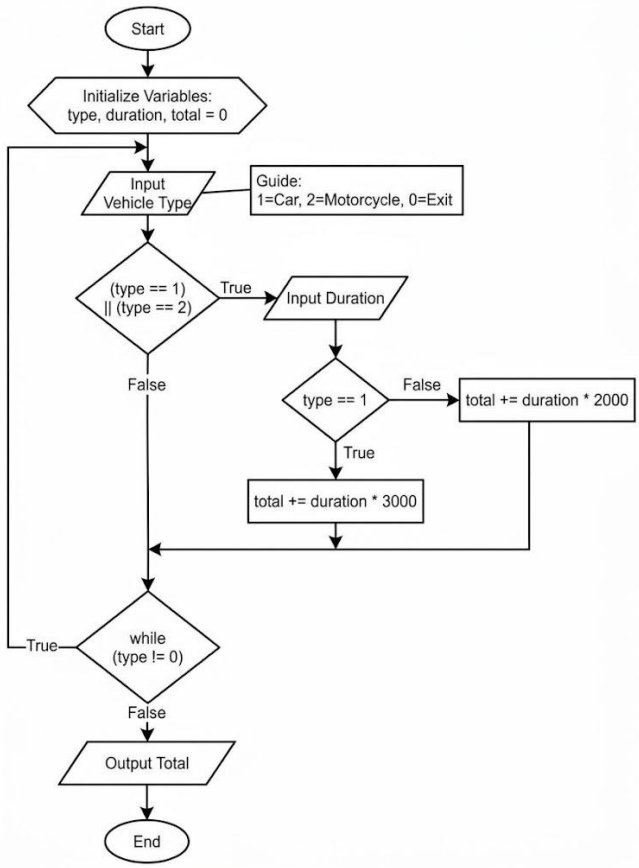
1. In this DO-WHILE loop, if the first customer name entered is “cancel”, how many times is the loop executed?
2. State the stopping condition used in this DO-WHILE loop!
3. What is the function of using the value **true** in the DO-WHILE condition?
4. Why does this DO-WHILE loop still run even though it has no initialization and update components?

3. Assignment

Duration: 120 minutes

1. A cinema manager wants to create a program to calculate the total ticket sales in one day. Tickets are sold at Rp 50,000 per ticket. The program must calculate the total number of tickets sold and the total ticket sales revenue for the day, subject to the following rules:
   - If a customer buys more than 4 tickets, the customer receives a 10% discount.
   - If a customer buys more than 10 tickets, the customer receives a 15% discount.
   - If the ticket quantity entered is invalid (negative), the program ignores the input and asks for it to be re-entered.

   Note: Loops may use for, while, or do-while. Add break or continue statements if necessary.
2. Refer to the following flowchart!



A parking lot operator wants to create a program to calculate the total parking fees of several vehicles. The parking rate is Rp 3,000 per hour for cars and Rp 2,000 per hour for motorcycles. The program keeps asking for input as long as the input is not 0. Implement the flowchart as a Java program!